Planning in Situation Calculus — Q1 Notes

The bare minimum vocabulary

Situation calculus encodes a world that changes over time inside first–order logic.

- **Situation** s: a snapshot of the world. s_0 is the initial situation.
- **Fluent**: a predicate whose truth value depends on a situation, e.g. $\textsc{At}(\text{robot}, \text{kitchen}, s)$.
- **Action**: an object, e.g. $\textsc{Go}(\text{kitchen}, \text{hall})$. Actions are *not* situations.
- **Result function**: $\textsc{Result}(a, s)$ is the situation obtained by performing action a in s.
- **Goal** $\textsc{Goal}(s)$: a fluent saying "the goal holds in situation s".
- **Sequence / Do**: $\textsc{Sequence}(a, s, s')$ (a.k.a. $\textsc{Do}$) means "executing the action list a starting in s produces s'". Defined recursively:

$$\begin{gathered} \textsc{Sequence}([\,],\ s,\ s) \quad \text{(empty list leaves } s \text{ unchanged)} \\ \textsc{Sequence}([a \mid \mathit{rest}],\ s,\ s') \iff \textsc{Sequence}(\mathit{rest},\ \textsc{Result}(a, s),\ s') \end{gathered}$$

Successor-state axioms (one per fluent) tell you when a fluent is true in $\textsc{Result}(a, s)$ in terms of a and s. They are what makes any such query evaluable, and they side-step the frame problem.

The two query forms

Form A — explicit situation

$$\exists a\, \exists s\, .\ \textsc{Sequence}(a, s_0, s) \ \wedge\ \textsc{Goal}(s)$$

"Is there an action list a and a situation s such that running a from s_0 produces s, and the goal holds in s?" A constructive proof binds a to a plan.

Form B — situation hidden

$$\exists \mathit{actionList}\, .\ \textsc{Goal}(\ldots \mathit{actionList} \ldots)$$

$\textsc{Goal}$ now takes the action list directly, with no explicit situation variable. We have to define what that means.

How to make Form B work — the key idea

We *redefine* $\textsc{Goal}$ so it takes an action list and *internally* threads it through s_0 using $\textsc{Result}$.

Construction. Define an auxiliary

$$\textsc{Do}([\,], s) = s, \qquad \textsc{Do}([a \mid \mathit{rest}], s) = \textsc{Do}(\mathit{rest}, \textsc{Result}(a, s)).$$

Now write the new goal predicate as

$$\textsc{Goal}(\mathit{actionList}) \iff \textsc{GoalHolds}\big(\textsc{Do}(\mathit{actionList},\ s_0)\big),$$

where $\textsc{GoalHolds}(s)$ is the original situation-based goal fluent. **The situation argument is folded into the list recursion.**

Equivalently, write $\textsc{Goal}$ recursively on the list (no helper):

$$\begin{aligned} \textsc{Goal}([\,]) &\iff \textsc{GoalHolds}(s_0) \\ \textsc{Goal}([a \mid \mathit{rest}]) &\iff \textsc{Goal}_{\mathit{after\text{-}step}}(a, \mathit{rest}) \end{aligned}$$

where $\textsc{Goal}_{\mathit{after\text{-}step}}$ peels one action off the list, applies $\textsc{Result}$ once, and recurses — which is exactly the same computation as before, just rolled into the definition of $\textsc{Goal}$.

What to write in the exam answer

A clean answer has **three moves**:

1. **Identify what is missing.** Form B has no situation variable, so $\textsc{Goal}$ cannot literally be the situation-based fluent of Form A. It must be re-defined to operate on the action list.
2. **Re-thread the situation.** Provide the recursive definition that walks the action list from s_0, applying $\textsc{Result}$ once per action, ending in some situation s_n. (Show the base case and recursive case — two lines.)
3. **State the equivalence.** $\textsc{Goal}(\mathit{actionList})$ then unfolds to $\textsc{GoalHolds}(\textsc{Do}(\mathit{actionList}, s_0))$, which holds iff Form A would have held with $s = \textsc{Do}(\mathit{actionList}, s_0)$. So Form B is solvable by exactly the same successor-state axioms; only the existential over s has been eliminated.

Why bother? The witness extracted from a constructive proof is now *just* the action list — which is what the planner actually wanted in the first place. Form A forces you to existentially quantify over situations (a richer, less convenient domain) even though the situation itself is a derived quantity.

Tiny worked sketch

Let $s_0 = \{\textsc{At}(R, \text{kitchen})\}$, goal $\textsc{At}(R, \text{hall})$, action $\textsc{Go}(x, y)$ with successor-state axiom giving $\textsc{At}(R, y, \textsc{Result}(\textsc{Go}(x, y), s))$ once preconditions hold.

Form B query: $\exists \mathit{actionList}.\ \textsc{Goal}(\mathit{actionList})$.

Try $\mathit{actionList} = [\textsc{Go}(k, h)]$. Unfolding:

$$\begin{aligned} \textsc{Goal}([\textsc{Go}(k, h)]) &\iff \textsc{GoalHolds}\big(\textsc{Result}(\textsc{Go}(k, h),\ s_0)\big) \\ &\iff \textsc{At}(R, h) \text{ in that situation} \quad \iff \quad \top. \end{aligned}$$

Witness: $\mathit{actionList} = [\textsc{Go}(\text{kitchen}, \text{hall})]$. □